Les Structures Répétitives en Algorithmique et Langage C : Boucles While, Do-While et For

Thème principal : Structures itératives, condition d'arrêt, boucles while, do-while et for en C

Synthèse exécutive : Ce cours aborde de manière approfondie les structures répétitives (boucles) en algorithmique et en langage C. L'objectif fondamental est d'automatiser le traitement répétitif d'instructions sans duplication de code, en stricte conformité avec le principe DRY (Don't Repeat Yourself). La séance détaille le fonctionnement, la syntaxe C et les critères de choix entre les trois boucles fondamentales : 'while' (à pré-condition), 'do-while' (à post-condition) et 'for' (à itération déterminée). Un accent particulier est mis sur la gestion rigoureuse des variables de contrôle pour éviter les boucles infinies, la portée des variables d'itération et la syntaxe C exacte (comme le point-virgule obligatoire en fin de do-while).

1. Utilité des structures itératives et principe DRY

En programmation, répétition ne doit jamais rimer avec duplication. Dupliquer manuellement des lignes de code pour traiter un tableau de 10 000 éléments ou redemander une saisie utilisateur invalide détruit la maintenabilité, alourdit inutilement le programme et génère une source majeure de bogues. Les structures itératives (ou boucles) permettent d'exécuter un même bloc d'instructions de manière répétée tant qu'une condition logique (expression booléenne) reste vraie. Pour qu'une boucle s'arrête, il est obligatoire que les instructions internes modifient l'état des variables testées dans la condition d'arrêt.

- Principe DRY (Don't Repeat Yourself) : Interdiction absolue de dupliquer du code pour exécuter plusieurs fois une tâche similaire.
- Condition d'arrêt : Expression booléenne évaluée à chaque itération qui détermine la poursuite ou la sortie de la boucle.
- Variable de contrôle / Compteur : Variable modifiée au sein du corps de la boucle pour faire évoluer l'expression booléenne vers Faux.
- Boucle infinie : Dysfonctionnement critique surviennent lorsque la condition d'arrêt reste indéfiniment Vraie (ex: omission de l'incrémentation).

Cas concret enseignant : *Métaphore du tapis roulant d'usine d'emballage : tant qu'un colis est détecté par le capteur (condition vraie), le bras robotisé attrape le colis et l'emballe (instruction). Si le capteur ne détecte plus de colis ou si le compteur atteint la limite fixée, le robot s'arrête. Sans condition de fin fonctionnelle, le robot continuerait à tourner à vide, ce qui entraînerait la surchauffe et la destruction de la machine.*

2. La boucle TANT QUE (while) : Structure à pré-condition

La structure 'while' s'impose lorsque le nombre exact d'itérations n'est pas connu à l'avance et dépend d'un événement dynamique (ex: une saisie utilisateur, la lecture d'un fichier jusqu me à la fin, ou la réception d'un signal réseau). Comme la condition est testée en amont, il convient d'être vigilant : si les variables de contrôle ne sont pas initialisées correctement avant le while, le comportement est indéterminé. De plus, l'absence de mise à jour de la variable de contrôle dans le corps du while provoque une boucle infinie qui bloque l'exécution du programme.

- Pré-conditionnement : La condition booléenne est évaluée AVANT chaque exécution du bloc d'instructions.
- Nombre d'itérations (0 à N) : Si la condition est fausse dès l'entrée, le bloc d'instructions n'est jamais exécuté (0 itération).
- Initialisation préalable : Toutes les variables intervenant dans la condition du while doivent obligatoirement posséder une valeur définie avant le test.

Cas concret enseignant : *Exemple du contrôle de code PIN au distributeur de billets : la machine demande le code TANT QUE le code saisi est incorrect ET QUE le nombre d'essais est inférieur à 3. Si l'usager tape le bon code dès le premier essai, la boucle s'arrête immédiatement sans répéter la demande.*

3. La boucle FAIRE... TANT QUE (do-while) : Structure à post-condition

Contrairement au 'while', la boucle 'do-while' force un premier passage systématique dans le corps d'instructions avant d'évaluer la condition d'arrêt. Elle est l'outil algorithmique par excellence pour les contrôles de saisie sécurisés (validation de bornes numériques) et la gestion de menus interactifs. En C, la syntaxe s'écrit `do { ... } while (condition);`. L'absence du point-virgule final constitue l'une des fautes de syntaxe les plus fréquemment sanctionnées lors des contrôles machine.

- Post-conditionnement : La condition booléenne est évaluée APRÈS l'exécution du bloc d'instructions.
- Garantie d'exécution (1 à N) : Le corps de la boucle est exécuté au minimum 1 fois, quel que soit l'état initial de la condition.
- Syntaxe C stricte : Obligation absolue d'apposer un point-virgule ';' immédiatement après la parenthèse fermante de la condition `while(condition);`.

Cas concret enseignant : *Contrôle de saisie d'une note d'examen : 'Il est techniquement impossible de vérifier si la saisie faite par un étudiant est valide avant qu'il n'ait tapé au moins une valeur sur le clavier. On doit donc exécuter la saisie au moins une fois, ce qui fait du do-while la structure naturellement adaptée.'*

4. La boucle POUR (for) : Itération déterminée

La structure 'for' regroupe en une ligne unique toute la mécanique d'itération. Les trois expressions séparées par des points-virgules dans l'en-tête sont : 1) l'initialisation de la variable de contrôle, 2) la condition logique d'exécution, et 3) la mise à jour (incrémentation ou décrémentation). Bien qu me le C autorise l'omission de certaines parties (ex: `for(;;)` qui crée une boucle infinie), les renseigner explicitement est indispensable pour la lisibilité et les exigences d'évaluation en BUT.

- En-tête condensé : Centralise l'initialisation, la condition de maintien et l'incrémentation sous la forme `for(init; cond; inc)`.
- Usage privilégié : Dédiée aux cas où le nombre d'itérations est fixe et connu avant d'entrer dans la boucle (ex: parcours de tableaux).
- Portée (scope) : En C99 et supérieur, la variable d'itération peut être déclarée directement dans l'en-tête `for(int i = 0; ...)`.

Cas concret enseignant : *Métaphore des tours de piste d'athlétisme : 'Vous savez avant de vous lancer que vous devez effectuer exactement 5 tours. Vous initialisez votre compteur à 1 (init), vous vérifiez que vous n'avez pas dépassé 5 (cond), et à chaque franchissement de la ligne d'arrivée, vous ajoutez 1 à votre compteur (inc).'*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi le programme plante ou bloque indéfiniment quand j'oublie d'incrémenter 'i' dans une boucle while ?
Réponse du professeur : Si vous n'incrémentez pas 'i' dans le corps de la boucle, la valeur de 'i' ne change jamais. La condition d'arrêt (par exemple i < 10) reste indéfiniment VRAIE. Le programme entre alors dans une 'boucle infinie', consommant 100% des ressources du processeur sans jamais pouvoir passer à l'instruction suivante.

Question étudiant : Est-ce qu'on peut toujours remplacer une boucle 'for' par une boucle 'while' ?
Réponse du professeur : Oui, sur le plan algorithmique, toute boucle 'for' peut être traduite par une boucle 'while' en déportant l'initialisation avant le while et l'incrémentation à la fin du corps de la boucle. Cependant, la boucle 'for' reste fortement recommandée lorsque le nombre d'itérations est connu à l'avance car elle est plus lisible et évite d'oublier l'incrémentation.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un programme complet en langage C qui calcule la somme des N premiers entiers positifs (ex: si N = 5, calculer 1 + 2 + 3 + 4 + 5 = 15). Le programme doit valider que N est un entier strictement positif (> 0) via une boucle 'do-while', puis calculer la somme au moyen d'une boucle 'for'.

Résolution pas à pas :
Étape 1 : Déclarer les variables entières n, somme et i. Étape 2 : Mettre en place la saisie sécurisée du nombre N avec la structure do-while : do { printf("Entrez N (>0) : "); scanf("%d", &n;); } while (n <= 0); Étape 3 : Initialiser la variable de cumul : somme = 0; Étape 4 : Utiliser une boucle for pour calculer la somme cumulative de 1 jusqu'à N : for (i = 1; i <= n; i++) { somme += i; } Étape 5 : Afficher le résultat avec printf("La somme des %d premiers entiers est %d\n", n, somme);

Résultat : Pour une saisie utilisateur où N = 5 : La boucle 'do-while' valide N (>0). La boucle 'for' réalise 5 itérations (i=1 -> somme=1 ; i=2 -> somme=3 ; i=3 -> somme=6 ; i=4 -> somme=10 ; i=5 -> somme=15). Le programme affiche correctement : 'La somme des 5 premiers entiers est 15'. Si l'utilisateur entre au départ une valeur négative ou nulle (ex: -3), la boucle do-while rejette la valeur et re-demande la saisie.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale d'évaluation entre une boucle 'while' et une boucle 'do-while' ?

R : La boucle 'while' est à pré-condition : elle évalue la condition AVANT chaque itération (de 0 à N passages). La boucle 'do-while' est à post-condition : elle évalue la condition APRÈS chaque itération, garantissant au moins 1 exécution du bloc d'instructions.

Q : Que se passe-t-il si la condition d'une boucle 'for' est fausse dès le premier test ?

R : Si la condition de l'en-tête du 'for' est fausse dès le premier contrôle initial, le corps de la boucle n'est jamais exécuté (0 itération) et le programme reprend immédiatement après la boucle.

Q : Quel oubli de syntaxe systématique provoque une erreur de compilation sur la boucle 'do-while' en langage C ?

R : L'oubli du point-virgule final ';' immédiatement après la parenthèse fermante de la condition while : `do { ... } while (condition);`.

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765